
MODULE – 5: Operating System Basics

RTOS and IDE for Embedded System Design: Operating System basics,

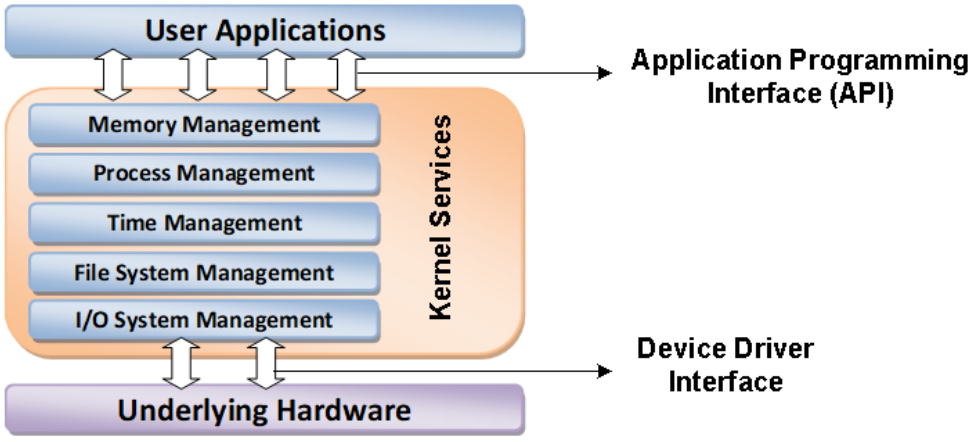
Types of operating systems, Task, process and threads (Only POSIX Threads with an example program), Thread preemption, Preemptive Task scheduling techniques, Task Communication, Task synchronization issues – Racing and Deadlock, Concept of Binary and counting semaphores (Mutex example without any program), How to choose an RTOS, Integration and testing of Embedded hardware and firmware, Embedded system Development Environment – Block diagram (excluding Keil), Disassembler/decompiler, simulator, emulator and debugging techniques **(Text 2: Ch-10 (Sections 10.1, 10.2, 10.3, 10.5.2 , 10.7, 10.8.1.1, 10.8.1.2, 10.8.2.2, 10.10 only), Ch-12, Ch-13 (a block diagram before 13.1, 13.3, 13.4, 13.5, 13.6 only)**

10 Hours

Operating System basics:

The operating system acts as a bridge between the user applications/tasks and the underlying system resources through a set of system functionalities and services. The OS manages the system resources and makes them available to the user applications/tasks on a need basis. The primary functions of an operating system are

- Make the system convenient to use
- Organise and manage the system resources efficiently and correctly



The Operating System Architecture

Kernel:

The kernel is the core of the operating system and is responsible for managing the system resources and the communication among the hardware and other system services. Kernel acts as the abstraction layer between system resources and user applications. It contains a set of system libraries and services. The kernel contains different services for handling the following.

Process Management:

Process management deals with managing the processes/tasks. Process management includes setting up the memory space for the process, loading the process's code into the memory space, allocating system resources, scheduling and managing the execution of the process, setting up and managing the Process Control Block (PCB), Inter Process Communication and synchronization, process termination/deletion, etc

Primary Memory Management:

The term primary memory refers to the volatile memory (RAM) where processes are loaded and variables and shared data associated with each process are stored. The Memory Management Unit (MMU) of the kernel is responsible for

• Keeping track of which part of the memory area is currently used by which process

• Allocating and De-allocating memory space on a need basis (Dynamic memory allocation).

File System Management:

File is a collection of related information. A file could be a program (source code or executable), text files, image files, word documents, audio/video files, etc. Each of these files differ in the kind of information they hold and the way in which the information is stored. The file operation is a useful service provided by the OS.

The file system management service of Kernel is responsible for

- The creation, deletion and alteration of files
- Creation, deletion and alteration of directories
- Saving of files in the secondary storage memory (e.g. Hard disk storage)
- Providing automatic allocation of file space based on the amount of free space available
- Providing a flexible naming convention for the files.

The various file system management operations are OS dependent. For example, the kernel of Microsoft® DOS OS supports a specific set of file system management operations and they are not the same as the file system operations supported by UNIX Kernel.

I/O System (Device) Management:

Kernel is responsible for routing the I/O requests coming from different user applications to the appropriate I/O devices of the system. In a well-structured OS, the direct accessing of I/O devices are not allowed and the access to them are provided through a set of Application Programming Interfaces (APIs) exposed by the kernel. The kernel maintains a list of all the I/O devices of the system in advance, at the time of building the kernel. Some kernels, dynamically updates the list of available devices as and when a new device is installed (e.g. Windows XP kernel keeps the list updated when a new plug "n" play USB device is attached to the system). The service 'Device Manager' (Name may vary across different OS kernels) of the kernel is responsible for handling all I/O device related operations. The kernel talks to the I/O device through a set of low-level systems calls, which are implemented in a service, called device drivers. The device drivers are specific to a device or a class of devices.

The Device Manager is responsible for

- Loading and unloading of device drivers

• Exchanging information and the system specific control signals to and from the device

Secondary Storage Management:

The secondary storage management deals with managing the secondary storage memory devices, if any, connected to the system. Secondary memory is used as backup medium for programs and data since the main memory is volatile. In most of the systems, the secondary storage is kept in disks (Hard Disk).

The secondary storage management service of kernel deals with

- Disk storage allocation
- Disk scheduling (Time interval at which the disk is activated to backup data)
- Free Disk space management

Protection Systems:

Most of the modem operating systems are designed in such a way to support multiple users with different levels of access permissions (e.g. Windows XP with user permissions like 'Administrator', 'Standard', 'Restricted', etc.). Protection deals with implementing the security policies to restrict the access to both user and system resources by different applications or processes or users. In multiuser supported operating systems, one user may not be allowed to view or modify the whole/ portions of another user's data or profile details. In addition, some application may not be granted with permission to make use of some of the system resources. This kind of protection is provided by the protection services running within the kernel.

Interrupt Handler:

Kernel provides handler mechanism for all extemal/intemal interrupts generated by the system.

Along with these important services the kernel of an operating system offers offer a number of add-on system components/services to the kernel. Network communication, network management, user-interface graphics, timer services (delays, timeouts, etc.), error handler, database management, etc.

Kernel Space and User Space:

The applications/ services are classified into two categories, namely: user applications and kernel applications. The program code corresponding to the kernel applications/services are called as kernel code. The contiguous area of primary (working) memory at which the kernel code are located are called as kernel space and is protected from the un-authorised access by user programs/applications. Similarly, all user applications are loaded and executed on a specific area of primary memory known as 'User Space'. The partitioning of memory into kernel and user space is purely Operating System dependent.

Types of Kernel:

Based on kernel design, kernel can be classified into Monolithic & Microkernel.

Monolithic Kernel:

- In Monolithic kernel architecture all the kernel service run in the kernel space – means all kernel module run with same memory space under single kernel thread

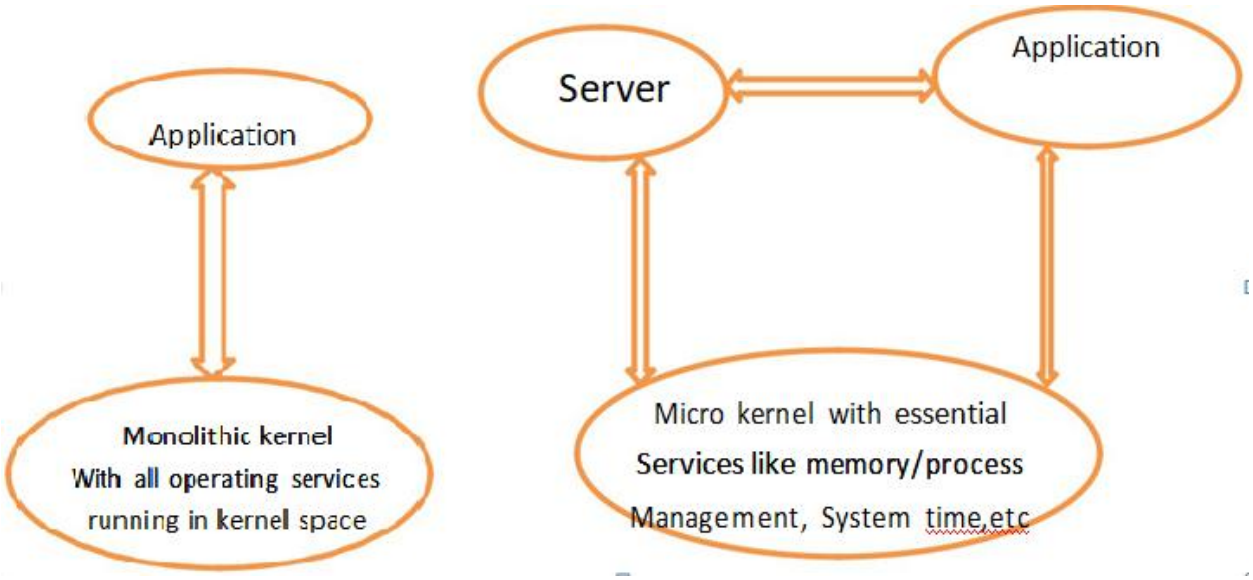
- The drawback of Monolithic kernel is that any error or failure in any of the kernel module i.e. leads to the crashing of entire kernel application. Ex :

  LINUX,SOLARIS,MS-DOS

Micro kernel:

Design incorporates only the essential set of OS services into the kernel. The rest of the OS services are implanted in programs known as "servers" which runs in user space.

The essential services of the Micro kernel are Memory management, Process management, Timer system, Interrupt handler. Ex. OS , MACH , QNX , MINIX3 Benefits of Micro kernel are Robust and Configurability.



Monolithic Kernel Model **Microkernel Model**

Types of Operating System

Depending on the type of kernel & kernel services purpose & type of in computing system OS are classified into two types.

1. General Purpose Operating System[GPOS]
2. Real Time Operating System[RTOS]

General Purpose Operating System[GPOS]

- The OS which are deployed in general computing systems are referred as general purpose OS.

- The kernel of such OS is more generalized & it contains all kinds of services required for executing generic applications

- GPOS are quite non-deterministic in execution behaviour

Ex: PC/Desktop system, windows XP & MS-DOS

Real Time Operating System [RTOS]

- There is no universal definition available for the term Real Time. Real-time implies deterministic timing behavior – means the OS services consumes only known & expected amount of time regardless the no of services.

- A Real-Time Operating System or RTOS implements policies and rules concerning time-critical allocation of a system's resources.

- The RTOS decides which application should run in which order & how much time needs to be allocated for each applications.

- A RTOS is an OS intended to serve real-time application requests. It must be able to process the data as it comes in typically without buffering delays.

- Processing time requirements are measured in tenth of seconds. Hard RTOS has less jitter than soft RTOS.

Ex: Windows CE, UNIX,VxWorks,Micros/OS-II

Real Time Kernel:

The RTOS is referred to as Real Time kernel in complement to conventional OS kernel. The kernel is highly specialized & it contains only the minimal set of services required for running the user application/ tasks.

The Basic Functions of Real Time Kernel are

1. Task/Process Management
2. Task/Process Scheduling
3. Task/Process Synchronization
4. Error/Exception Handling
5. Memory Management
6. Interrupt Handling
7. Time Management

Task/Process Management:

Task/Process management deals with setting up the memory space for the tasks, loading the tasks code into memory space, allocating system resources, setting up a Task Control Block [TCB] for the task & task/process termination/deletion.

TCB: Task Control Block is used for holding the information corresponding to a task. The TCB contains the following set of information.

- Task ID: Task identification number
- Task State: The current state of the task
- Task Type: Indicates what is the type of this task, the task can be hard real time or soft real time or background task
- Task Priority: Ex task priority=1 for task with priority =1
- Task Context Pointer: context pointer, pointer for saving context
- Task Memory Pointer: Pointers to the code memory, data memory & stack memory for the task.
- Task System Resource Pointers: Pointer to system resource used by the task.
- Task Pointers: Pointer to other TCB's.
- Other Parameters: Other relevant task parameters.
- The TCB parameters vary across different kernels based on the task management Implementation. TCB is created on task creation and deleted/Removed when a task is terrminated or deleted.TCB is read to get the state of a task and its parameters are updated on need basis. TCB is modified to change the priority of task dynamically

Task/Process Scheduling:

Process scheduling deals with sharing the CPU among various tasks/process. A kernel applications called scheduler, handles the task scheduling. Scheduler is nothing but an algorithm implementation, which performs the efficient and optimal scheduling of tasks to provide a deterministic behaviour.

Task/Process Synchronization:

Task/Process Synchronization deals with synchronising the concurrent access of a resource, which is shared across multiple tasks and the communication between various tasks.

Error/Exception Handling:

Deals with registering& handling the errors occurred during the execution of tasks. Ex: Insufficient memory, time outs, deadlocks, dead line missing, bus error, divide by zero, unknown instruction execution. Errors & exceptions can happen at Kernel Level Service and

at Task Level services. Deadlock is an example for kernel level exception, whereas timeout is an example for a task level exception. The OS kernel gives the information about the error in the form of a system call (API). GetLastError() API provided by Windows CE RTOS is an example for such a system call.

Memory Management:

RTOS makes use of 'Block Based Memory' allocation techniques instead of the usual dynamic memory allocation technique used by GPOS. RTOS kernel uses blocks of fixed size dynamic memory & block is allocated for a task on a need of basis. A few RTOS kernel implements Virtual Memory concepts avoid the garbage collection overhead.

Interrupt Handler:

Interrupt handler deals with handling several interrupts. Interrupts provide Real Time behavior to systems. Interrupts inform the processor that an external device or an associated task required immediate attention of the CPU. Interrupts can be either Synchronous Asynchronous. Interrupts which occurs in synch with the currently executing task is known as synchronous interrupts. Usually the system interrupts fall under the synchronous category Ex Divide by zero, memory segmentation error. A synchronous interrupts are those which occurs at any point of execution of any task & are not in sync with currently executing tasks.

Time Management:
Accurate time management is essential for providing precise time reference for all applications. The time reference to kernel is provided by a high-resolution Real-Time Clock (RTC) hardware chip (hardware timer). The hardware timer is programmed to interrupt the processor/controller at a fixed rate called 'Timer tick'. The 'Timer tick' is taken as the timing reference by the kernel. The 'Timer tick' interval usuall in microseconds range may vary depending on the hardware timer. The time parameters for tasks are expressed as the multiples of the 'Timer tick'. The System time is updated based on the 'Timer tick'.

Hard Real-Time Systems:

Real-Time Operating Systems that strictly adhere to the timing constraints for a task are referred as 'Hard Real-Time' systems. A Hard Real-Time system must meet the deadlines for a task without any slippage. Missing any deadline may produce catastrophic results for Hard Real-Time Systems], including permanent data lose and irrecoverable damages to the system/Users. Air bag control systems and Anti-lock Brake Systems (ABS) of vehicles are typical examples for Hard Real-Time Systems.

Soft Real-Time Systems:

Real-Time Operating System that does not guarantee meeting deadlines, but offer the best effort to meet the deadline are referred as 'Soft Real-Time' systems. Missing deadlines for tasks are acceptable for a Soft Real-time system if the frequency of deadline missing is within the compliance limit of the Quality of Service (QoS). Automatic Teller Machine (ATM) and Audio/Video play back System are typical examples for Soft- Real-Time System.

Task, Process & Threads

Task:

Refers to something that needs to be done. The tasks refers to the one assigned by our managers or the one assigned by our professors/teachers. In OS context, a task is defined as the program in execution & related information maintained by the OS. Task is also known as "JOB" in the OS context.

Process:

A process is a program or part of the program in execution. Process is also known as instance of the program in execution. Multiple instances of the same program can execute simultaneously. Process requires various system resources like CPU for executing the process, memory for storing the code corresponding to the process, I/O devices for information exchange etc.

The Structure of a Process:

The concept of 'Process' leads to concurrent execution (pseudo parallelism) of tasks and thereby the efficient utilisation of the CPU-and other system resources. Concurrent execution is achieved through the sharing of CPU among the processes. A process mimics a processor in properties and holds a set of registers, process status, a Program Counter (PC) to point to the next executable instruction of the process, a stack for holding the local variables associated with the process and the code corresponding to the process. This can be visualised as shown in Fig. below.

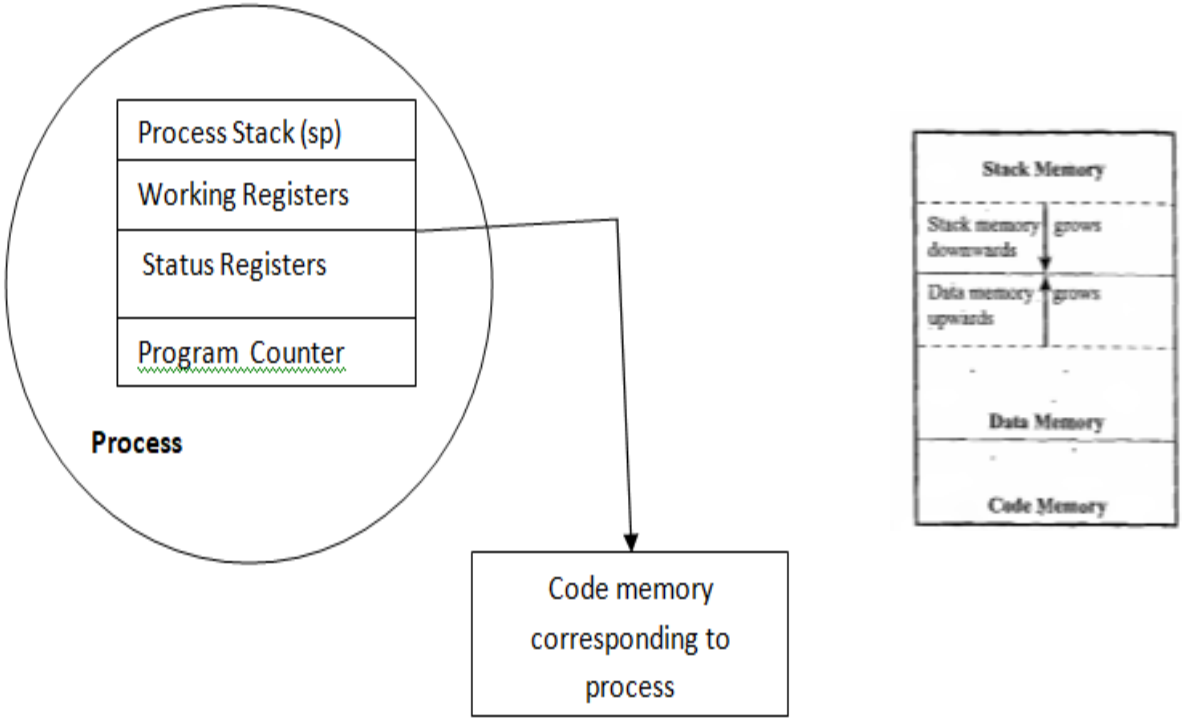


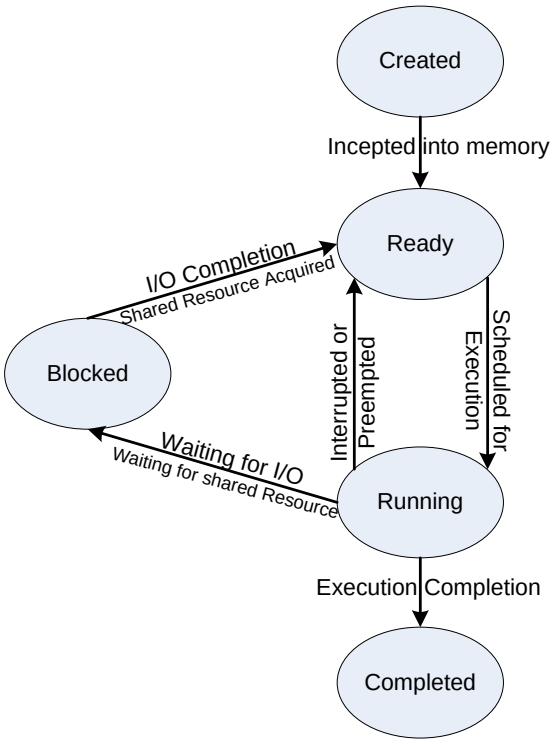
Fig: Process Structure

A process which inherits all the properties of the CPU can be considered as a virtual processor, awaiting its turn to have its properties switched into the physical processor. When the process gets its turn, its registers arid the program counter register becomes mapped to the physical registers of the CPU. From a memory perspective, the memory occupied by the process is segregated into three regions, namely, Stack memory, Data memory and Code memory. The 'Stack' memory holds all temporary data such as variables local to the process. Data memory holds all global data for the process. The code memory contains the program code (instructions) corresponding to the process. On loading a process into the main memory, a specific area of memory is allocated for the process.

Process management:

Process management deals with the creating a process, setting up the memory space for the process, loading the process code with the memory space, allocating the system resources. Setting up a process control block (PCB) for the process and process termination/detection.

Process States & State Transition



Created State:

The state at which a process is being created is referred as ‘Created State’. The Operating System recognises a process in the ‘‘Created State' but no resources are allocated to the process.

Ready State:

The state where a process is incepted into the memory and awaiting for the processor time for execution, is known as ‘Ready State'. At this stage, the process is placed in the ‘Ready list' queue maintained by the OS.

RunningState:

The state where in the source code instructions corresponding to the process is being executed is called ‘Running State'. Running state is the state at which the process execution happens.

Blocked State:

Blocked State/Wait State' refers to a state where a running process is temporarily suspended from execution and does not have immediate access to resources. The blocked state might be invoked by various conditions like: the process enters a wait state for an event to occur (e.g. Waiting for user inputs such as keyboard input) or waiting for getting access to a shared resource.

Completed State:

A state where the process completes its execution is known as 'Completed State'.

The transition of a process from one state to another is known as 'State transition'. When a process changes its state from Ready to running or from running to blocked or terminated or from blocked to running, the CPU allocation for the process may also change

Thread

A thread is the primitive that can be executed code. A thread is a single sequential flow of control within process. Thread is also known as light weight process. A process can have many threads of execution. Different threads which are part of a process share the same data memory, code memory and heap memory area of a process. Threads maintain their own thread status (CPU register values), Program Counter (PC) and stack. The memory model for a process and its associated threads are given in fig.

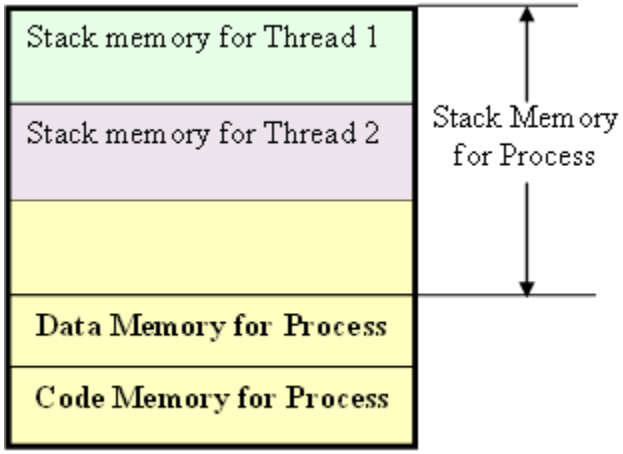


Fig. Memory Organisation of a process and its associated threads

Multithreading:

A process/task in embedded application may be a complex or lengthy one and it may contain various sub operations like getting input from I/O devices connected to the processor, performing some internal calculations/operations, updating some I/O devices etc. If all the subfunctions of a task are executed in sequence, the CPU utilisation may not be efficient. Instead of this single sequential execution of the whole process, if the task/process is split into different threads carrying out the different subfunctionalities of the process, the CPU can be effectively utilised and when the thread corresponding to the I/O operation enters the wait state, another threads which do not require the I/O event for their operation can be switched into execution. This leads to more speedy execution of the process and the efficient utilisation of the processor time and resources.

If the process is split into multiple threads, which executes a portion of the process, there will be a main thread and rest of the threads will be created within the main thread.

Use of multiple threads to execute a process brings the following advantage.

• Better memory utilisation.

• Speeds up the execution of the process.

• Efficient CPU utilisation.

Win 32, java.threads, posix threads are the different Thread standard supported by different OS.

POSIX Threads:

POSIX stands for Portable Operating System Interface. The POSIX standard deals with the Real-Time extensions and POSIX.4a standard deals with thread extensions. The POSIX standard library for thread creation and management is ' Pthreads'. ‘P threads' library defines the set of POSIX thread creation and management functions in ‘C’ language.

The primitive

int pthread_create(pthread_t.*new_thread_ID, const pthread_attr_t *attribute, void * (*start_function)(void *), void *arguments); creates a new thread for running the function start_ function. pthread is the handle to the newly created thread and pthread_attrj is the data type for holding the thread attributes, 'start_function’ is the function the thread is going to execute and arguments is the arguments for 'start Junction’ (It is a void * in the above example). On successful creation of a Pthread, pthread_create associates the Thread Control Block (TCB) corresponding to the newly created thread to the variable of type pthreadj (newjhreadJD in our example).

The primitive

int pthread_join (pthread_t new_thread,void * *thread_status); blocks the current thread and waits until the completion of the thread pointed by it. All the POSIX ‘thread calls’ returns an integer. A return value of zero indicates the success of the call. It is always good to check the return value of each call.

Write a multithreaded application to print “Hello I’m in main thread” from the main thread and “Hello I’m in new thread” 5 times each, using the pthread_create and pthreadJoin POSIX primitives.

```
# include <pthread.h>

# include <stbiib.h>

# include <stdio.h> ~J

/**********.***************/
```

New thread function for printing "Hello I'm in new thread"

```
void *new_thread ( void *thread_args )
{
int i, j;
for ( j= 0; j .< 5; j++ )
{
printf("Hello I'm in new thread\n" );
for ( i= 0; i < 10000; i+f ) ;
}
return NULL;
}
int main ( void )
{ int i, j;
If (pthread_create ( &tcb, NULL, new_thread, NULL );
{
printf ("Error in creating new threadin" ) ;
return -1;
 }
for ( j= 0; j < 5; j++)
{
printf ("Hello I'm in main thread" )
for ( i= 0; i < 10000; i++) ;
}
if (pthreadjoin (tcb, NULL ) )
{
printf ("Error in. Thread join"' ) ;
return -1;
 }
return 1;
 }
```

Task scheduling:

Multitasking involves the switching of execution among the different tasks. There should be same mechanism in place to show the CPU among the different tasks and to decide which process/task is to be executed at a given point of time. Determining which task/process is to be executed at a give point of time is known as task/process scheduling.Task scheduling forms the basis of multitasking. Scheduling policies forms the guidelines for determining which and when a task is to be executed.

A scheduling policy defines how processes are selected for promotion from the ready state to the running sate.

The process of scheduling may take place when a process switches its states to Ready state from Running state, Blocked/wait state from Running state, Ready state from Blocked/wait state, Completed (executed) state.

Scheduling can be either pre emptive or non-preemptive
When a high priority process in the blocked/wait state complete it's I/O and switches to ready state. The scheduler picks it for execution if the scheduling policy used is priority based preemptive.

The selection of scheduling algorithm should consider the following factors

CPU Utilization: Direct measure of how much % of CPU being utilized. A good scheduling algorithm should make the CPU utilization high.

Throughput: No. of processes executed per unit of time. Throughput of the scheduler should always be higher.

Turn around time: Time taken by a process to complete its execution. It includes the time spent by a process to wait for main memory, time spent in ready queue, time spent on completing the I/O opeartions and the time spent in execution.

Waiting Time: Time spent by process in ready queue.

Response time: Time elapsed between the submission of a process and the first response.

Scheduling algorithm are classified as (i) Pre-emptive scheduling (ii) Nonpre-emptive scheduling

Pre-emptive Scheduling: Moving the current running process into ready queue based on the priority or execution time of the newly entered process in ready state.

(i) Pre-emption based on timing
1. SJF pre-emptive scheduling

2.Round robin scheduling

(ii) Pre-emption based on priority
1. Pre-emptive priority scheduling.

1. **SJF pre-emptive scheduling**
   Processes in the ready queue are scheduled based on the execution time of the process. Process with minimum execution time will be scheduled first. When a new process with less execution time than the remaining available time of the current running process, then the current running process gets preempted and the newly entered process gets the CPU.
2. **Round robin scheduling**
   All the processes in the ready queue are scheduled as per the order in which the process enter the ready queue for predefined time slot. The first process gets the CPU for predefined time, then the second process gets the CPU for the predefined time. After scheduling all the process in the ready queue for predefined period of time slice, the same is repeated from the first process again.
3. **Pre-emptive priority scheduling.**

Processes in ready queue are scheduled based on the priority of the priority process. Process with highest priority gets the CPU first, then the next priority process and so on. If a new process with higher priority than the priority of the current running process enters the ready queue, then the current running process gets pre-empted and the newly entered process gets the CPU.

Note: Refer class notes for example and problems for each scheduling algorithm.

TASK COMMUNICATION

In a multitasking system, multiple tasks/processes run concurrently (in pseudo parallelism) and each process may or may not interact between. Based on the degree of interaction, the processes running on an OS are classified as

Co-operating Processes: In the co-operating interaction model one process requires the inputs from other processes to complete its execution.

Competing Processes: The competing processes do not share anything among themselves but they share the system resources. The competing processes compete for the system resources such as file, display device, etc.

Co-operating processes exchanges information and communicate through the following methods.

Co-operation through Sharing: The co-operating process exchange data through some shared resources.

Co-operation through Communication: No data is shared between the processes. But they communicate for synchronisation.

Task Communication through Shared memory

Processes share some area of the memory to communicate among them. Information to be communicated by the process is written to the shared memory area. Other processes which require this information can read the same from the shared memory area.The implementation of shared memory concept is kernel dependent. Different mechanisms are adopted by different kernels for implementing this. A few among them are:

Pipes:

'Pipe' is a section of the shared memory used by processes for communicating. Pipes follow the client-server architecture. A process which creates a pipe is known as a pipe server and a process which connects to a pipe is known as pipe client. A pipe can be considered as a conduct for information flow and has two conceptual ends. It can be unidirectional, allowing information flow in one direction or bidirectional allowing bi-directional information flow. A unidirectional pipe allows the process connecting at one

end of the pipe to write to the pipe and the process connected at the other end of the pipe to read the data, whereas a bi-directional pipe allows both reading and writing at one end . The unidirectional pipe can be visualised as

The implementation of 'Pipes' is also OS dependent. Microsoft Windows Desktop Operating Systems support two types of 'Pipes' for Inter Process Communication. They are:

Anonymous Pipes:

The anonymous pipes are unnamed, unidirectional pipes used for data transfer between two processes.

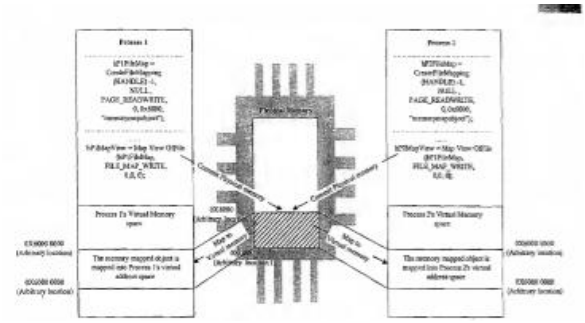
Named Pipes:

Named pipe is a named, unidirectional or bi-directional pipe for data exchange between processes. Like anonymous pipes, the process which creates the named pipe is known as pipe server. A process which connects to the named pipe is known as pipe client. With named pipes, any process can act as both client and server allowing point-to-point communication. Named pipes can be used for communicating between processes running on the same machine or between processes running on different machines connected to a network.

Memory Mapped Objects:

Memory mapped object is a shared memory technique adopted by certain Real-Time Operating Systems for allocating a shared block of memory which can be accessed by multiple process simultaneously. In this approach a mapping object is created and physical storage for it is reserved and committed. A process can map the entire committed physical area or a block of it to its virtual address space. All read and write operation to this virtual address space by a process is directed to its committed physical area. Any process which wants to share data with other processes can map the physical memory area of the mapped object to its virtual memory space and use it for sharing the data.

CreateFileMapping (HANDLE hFile,LPSECURITY_ATTRI- BUTES IpFileMappingAttributes, DWORD flProtect, DWORD dwMaximumSizeHigh, DWORD dw- MaximumSizeLow, LPCTSTR IpName) system call is used for sharing the memory.



The **MapViewOfFile** (HANDLE hFile, MappingObject DWORD dwDesiredAccess, DWORD dwFileOffsetHigh, DWORD dwFileOffsetLow, DWORD dwNumberOfBytesToMap) system call maps a view of the memory mapped object to the address space of the calling process.
If **dwNumberOfBytesToMap** is zero, the entire memory area owned by the memory mapped object is mapped. On successful execution, MapViewOfFile call returns the starting address of the mapped view.

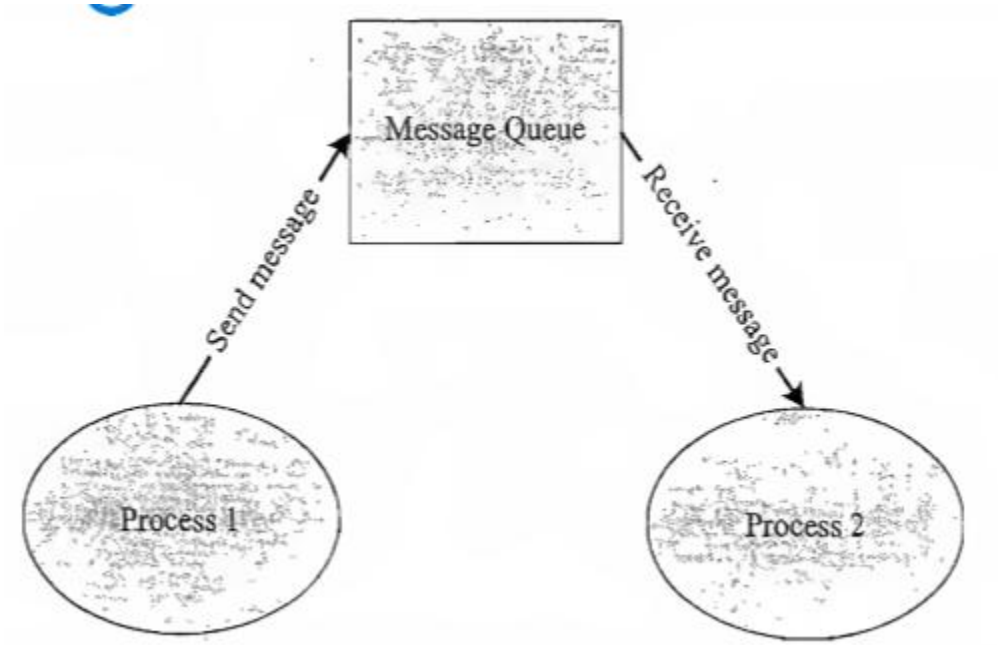
Message Passing

Message passing is an asynchronous information exchange mechanism used for Inter Process/Thread Communication. Unlike shared memory only limited amount of info/data is passed through message passing. Message passing is relatively fast and free from the synchronisation overheads compared to shared memory. Based on the message passing operation between the processes, message passing is classified into message Queue and mailbox.

Message Queue:

- The process which wants to talk to another process posts the message to a First-In-First-Out (FIFO) queue called 'Message queue', which stores the messages temporarily in a system defined memory object, to pass it to the desired process.
- The messages are exchanged through a message queue.
- The implementation of the message queue, send and receive methods are OS kernel dependent.
- The Windows XP OS kernel maintains a single system message queue and one process/thread specific'message queue.
- A thread which wants to communicate with another thread posts the message to the system message queue.
- The kernel picks up the message from the system message queue one at a time and examines the message for finding the destination thread and then posts the message to the message queue of the corresponding thread.
- Messages are sent and received through **send (Name of the process to which the message is to be sent, message) and receive (Name of the process from which the message is to be received, message)** methods.

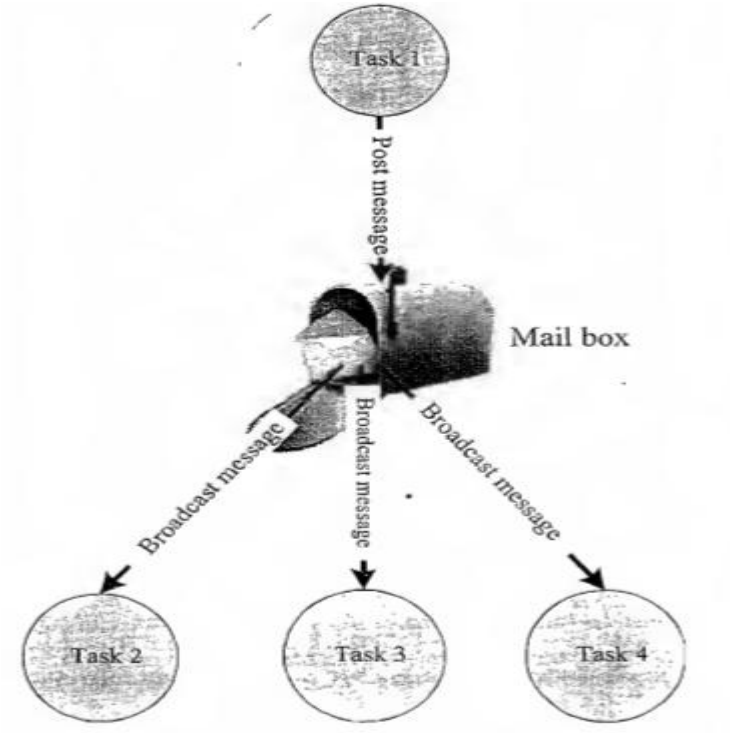
- A thread can simply post a message to another thread and can continue its operation or it may wait for a response from the thread to which the message is posted.
- The messaging mechanism is classified into synchronous and asynchronous based on the behaviour of the message posting thread.
- In asynchronous messaging, the message posting thread just posts the message to the queue and it will not wait for an acceptance (return) from the thread to which the message is posted.
- In synchronous messaging, the thread which posts a message enters waiting state and waits for the message result from the thread to which the message is posted. The thread which invoked the send message becomes blocked and the scheduler will not pick it up for scheduling.



MailBox

- Mailbox technique for IPC in RTOS is usually used for one way messaging.
- The task/thread which wants to send a message to other tasks/threads creates a mailbox for posting the messages.
- The thread which creates the mailbox is known as ‘mailbox server’.
- The threads which are interested in receiving the messages posted to the mailbox by the mailbox server can subscribe to the mailbox and these threads are known as ‘mailbox clients’.

- The mailbox server posts messages to the mailbox and notifies it to the clients which are subscribed to the mailbox.
- The clients read the message from the mailbox on receiving the notification.
- The mailbox creation, subscription, message reading and writing are achieved through OS kernel provided API calls.
- Mailbox and message queues differ in the number of messages supported by them. Mailbox is used for exchanging a single message between two tasks or between an Interrupt Service Routine (ISR) and a task.
- Mailbox associates a pointer pointing to the mailbox and a wait list to hold the tasks waiting for a message to appear in the mailbox. The implementation of mailbox is OS kernel dependent.



TASK SYNCHRONISATION

In a multitasking environment, multiple processes run concurrently (in pseudo parallelism) and share the system resources. The act of making processes aware of the access of shared resources by each process to avoid conflicts is known as 'Task/Process Synchronisation'. Various synchronisation issues may arise in a multitasking

environment if processes are not synchronised properly. The major task communication synchronisation issues in multitasking environment are

(i) Racing

(ii) Deadlocks

Racing:

Racing or Race condition is the situation in which multiple processes compete (race) each other to access and manipulate shared data concurrently. In a Race condition the final value of the shared data depends on the process which acted on the data finally. The best way to avoid racing is to make the access and modification of shared variables mutually exclusive; meaning when one process accesses a shared variable, prevent the other processes from accessing it

Racing synchronisation issue can be explained using the following example code

```
#include <windows.h>

#include < s tdio.h>

//counter is an integer variable and Buffer is a byte array shared between two //processes Process A and Process B

Char Buffer [10] = {1,2,3,4,5,6,7,8,9,10};

short int counter = 0;

// Process A

void Process A(void)

{

int i;

for (i =0; i<5; i++)

{

if (Buffer [i] >0)

counter++;

}}

// Process B

void Process B(void)

{
```

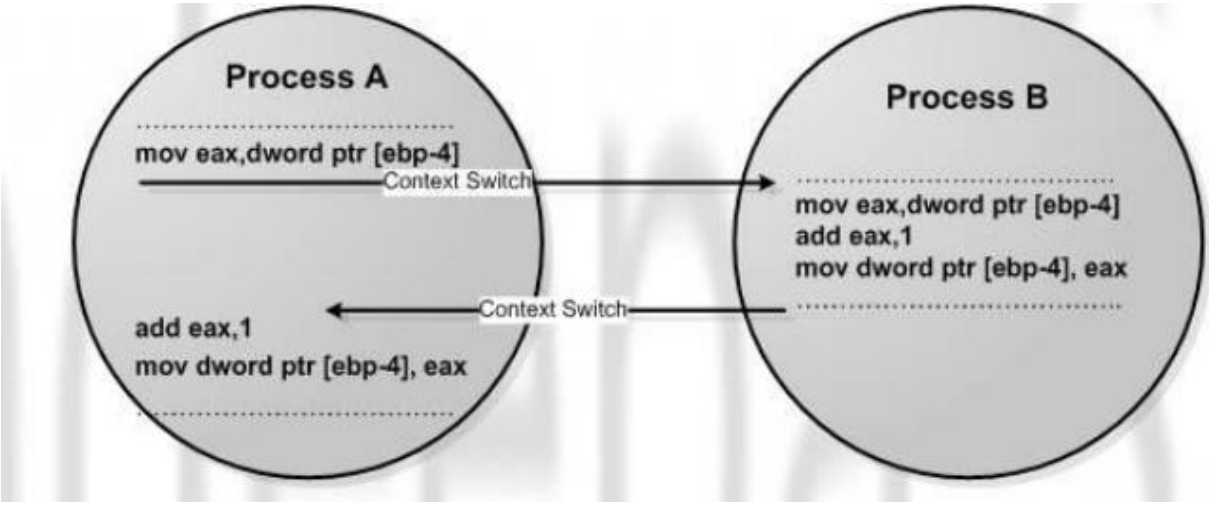
```
int j;
for (j =5; j<10; j++)
{ if (Buffer[j] > 0)
counter++;
}}
//Main Thread
int main ()
{
DWORD id;
CreateThread(Null, 0,(LPThREAD_START_ROUTINE) Process A,(LPVoid),0,0,&id);
CreateThread(Null, 0,(LPThREAD_START_ROUTINE) Process B,(LPVoid),0,0,&id);
Sleep(100000);
Return0;
}
```

From a programmer perspective the value of counter will be 10 at the end of execution of processes A & B. But 'it need not be always' in a real world execution of this piece of code under a multitasking kernel. The results depend on the process scheduling policies adopted by the OS kernel. The low level implementation of the high level program statement **counter++;** in the above code under Windows XP operating system running on an Intel Centrino Duo processor is given below

```
mov eax,dword ptr [ebp-4];Load counter in Accumulator
add eax,1 ; Increment Accumulator by 1
mov dword ptr [ebp-4],eax ;Store counter with Accumulator
```

At the processor instruction level, the value of the variable counter is loaded to the Accumulator register (EAX register). The memory variable counter is represented using a pointer. The base pointer register (EBP register) is used for pointing to the memory variable counter. After loading the contents of the variable counter to the Accumulator, the Accumulator content is incremented by one using the add instruction. Finally the content of Accumulator is loaded to the memory location which represents the variable counter. Both the processes Process A and Process B contain the program statement counter++;

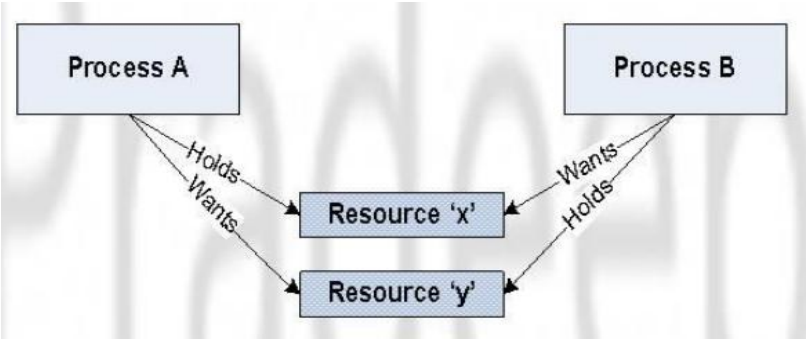
Imagine a situation where a process switching (context switching) happens from Process A to Process B when Process A is executing the counter++; statement. Process A accomplishes the counter++; statement through three different low level instructions. Now imagine that the process switching happened at the point where Process A executed the low level instruction, 'mov eax,dword ptr [ebp-4]' and is about to execute the next instruction 'add eax,l'. The scenario is illustrated in Fig.



Process B increments the shared variable 'counter' in the middle of the operation where Process A tries to increment it. When Process A gets the CPU time for execution, it starts from the point where it got interrupted (If Process B is also using the same registers eax and ebp for executing counter++; instruction, the original content of these registers will be saved as part of the context saving and it will be retrieved back as part of context retrieval, when process A gets the CPU for execution. Hence the content of eax and ebp remains intact irrespective of context switching). Though the variable counter is incremented by Process B, Process A is unaware of it and it increments the variable with the old value. **This leads to the loss of one increment for the variable counter. This problem occurs due to non-atomic! operation on variables.** This issue wouldn't have been occurred if the underlying actions corresponding to the program statement counter++; is finished in a single CPU execution cycle.

Deadlock:

Deadlock' is the condition in which a process is waiting for a resource held by another process which is waiting for a resource held by the first process as shown in figure.



Process A holds a resource x and it wants a resource y held by Process B. Process B is currently holding resource y and it wants the resource x which is currently held by Process A. Both hold the respective resources and they compete each other to get the resource held by the respective processes. The result of the competition is 'deadlock'. None of the competing process will be able-to access the resources held by other processes since they are locked by the respective processes

The different conditions favouring a deadlock situation are listed below.

Mutual Exclusion: The criteria that only one process can hold a resource at a time. Meaning processes should access shared resources with mutual exclusion. Typical example is the accessing of display hardware in an embedded device.

Hold and Wait: The condition in which a process holds a shared resource by acquiring the lock controlling the shared access and waiting for additional resources held by other processes.

No Resource Preemption: The criteria that Operating system cannot take back a resource from a process which is currently holding it and the resource can only be released voluntarily by the process holding it.

Circular Wait: A process is waiting for a resource which is currently held by another process which in turn is waiting for a resource held by the first process. In general, there exists a set of waiting process P0, PI ... Pn with P0 is waiting for a resource held by PI and PI is waiting for a resource held by PO, ..., Pn is waiting for a resource held by P0 and P0 is waiting for a resource held by Pn and so on... This forms a circular wait queue.

The OS may adopt any of the following techniques to detect and prevent deadlock conditions.

Ignore Deadlocks: Always assume that the system design is deadlock free. This is acceptable for the reason the cost of removing a deadlock is large compared to the chance of happening a deadlock. UNIX is an example for an OS following this principle. A life critical system cannot pretend that it is deadlock free for any reason.

Detect and Recover: This approach suggests the detection of a deadlock situation and recovery from it.Operating systems keep a resource graph in their memory. The resource graph is updated on each resource request and release. A deadlock condition can be detected by analysing the resource graph by graph analyser algorithms. Once a deadlock condition is detected, the system can terminate a process or preempt the resource to break the deadlocking cycle.

Avoid Deadlocks: Deadlock is avoided by the careful resource allocation techniques by the Operating System.

Prevent Deadlocks: Prevent the deadlock condition by negating one of the four conditions favouring the deadlock situation.

• Ensure that a process does not hold any other resources when it requests a resource.

• Ensure that resource preemption (resource releasing) is possible at operating system level.

Semaphores: Binary and Counting Semaphore

Semaphore is a sleep and wakeup based mutual exclusion implementation for shared resource access.

Semaphore is a system resource and the process which wants to access the shared resource can first acquire this system object to indicate the other processes which wants the shared resource that the shared resource is currently acquired by it.

The resources which are shared among a process can be either for exclusive use by a process or for using by a number of processes at a time.

Based on the implementation of the sharing limitation of the shared resource, semaphores are classified into two; namely ‘Binary Semaphore’ and ‘Counting Sema¬ phore’.

The binary semaphore provides exclusive access to shared resource by allocating the resource to a single process at a time and not allowing the other processes to access it when it is being owned by a process. Eg. Display device of an embedded system as a shared resource.

The implementation of binary semaphore is OS kernel dependent. Under certain OS kernel it is referred as mutex.

The ‘Counting Semaphore’ limits the access of resources by a fixed number of processes/threads. ‘Counting Semaphore’ maintains a count between zero and a value. It limits the usage of the resource to the maximum value of the count supported by it.

The state of the counting semaphore object is set to ‘signalled’ when the count of the object is greater than zero.

The count associated with a 'Semaphore object' is decremented by one when a process/thread acquires it and the count is incremented by one when a process/thread releases the 'Semaphore object'.

The state of the 'Semaphore object' is set to non-signalled when the semaphore is acquired by the maximum number of processes/threads that the semaphore can support.

The Hard disk (secondary storage) of a system is a typical example for sharing the resource among a limited number of multiple processes through counting semaphore. The creation and usage of 'counting semaphore object' is OS kernel dependent.

HOWTO CHOOSE AN RTOS

An RTOS for an embedded design is chosen by analysing some functional and non functional requirements.

Functional Requirements:

(i) Processor Support : It is not necessary that all RTOS's support all kinds of processor architecture. It is essential to ensure the processor support by the RTOS.

(ii) Memory Requirements: The OS requires ROM memory (FLASH) for holding the OS files and working memory RAM for loading the OS services. Since embedded systems are memory constrained, it is essential to evaluate the minimal ROM and RAM requirements for the OS under consideration.

(iii) Real-time Capabilities: It is not mandatory that the operating system for all embedded systems need to be Real-time. An embedded Operating system with Real-time behaviour is needed for an embedded system whose response requirements are high.

(iv) Inter Process Communication and Task Synchronisation: Certain kernels implement Inter Process Communication and Synchronisation policies for avoiding priority inversion issues in resource sharing.

(v) Modularisation Support : It is very useful if the OS supports modularisation where in which the developer can choose the essential modules and re-compile the OS image for functioning. Windows CE is an example for a highly modular operating system.

(vi) Support for Networking and Communication

(vii) Development Language Support : The OS may include the components like JVM (for running java applications) and .NET compact Framework(.NETCF for running .NET applications) as built-in component, if riot, check the availability of the same from a third party vendor for the OS under consideration.

Non-functional Requirements

(i) Custom Developed or Off the Shelf
(ii) Cost
(iii) Development and Debugging Tools Availability
(iv) Ease of Use

Multiprocessing, Multi programming & Multitasking

The ability to execute multiple processes simultaneously is referred as multiprocessing. Systems which are capable of performing multiprocessing are known as multiprocessor systems. Multiprocessor systems possess multiple CPUs and can execute multiple processes simultaneously

The ability of the Operating System to have multiple programs in memory, which are ready for execution, is referred as multiprogramming

Multitasking refers to the ability of an operating system to hold multiple processes in memory and switch the processor (CPU) from executing one process to another process. Multitasking involves 'Context switching', 'Context saving' and 'Context retrieval'. Context switching refers to the switching of execution context from task to other

When a task/process switching happens, the current context of execution should be saved to (Context saving) retrieve it at a later point of time when the CPU executes the process, which is interrupted currently due to execution switching. During context switching, the context of the task to be executed is retrieved from the saved context list. This is known as Context retrieval .

Simulators and Emulators

Simulators and emulators are two important debugging tools used in embedded system development.

Simulator

Simulator is a software tool used for simulating the various conditions for checking the functionality of the application firmware. Simulators simulate the target hardware and the firmware execution can be inspected using simulators. The features of simulator based debugging are listed below.

1. Purely software based
2. Doesn't require a real target system
3. Very primitive (Lack of featured I/O support. Everything is a simulated one)
4. Lack of Real-time behaviour

The Integrated Development Environment (IDE) itself will be providing simulator support and they help in debugging the firmware for checking its required functionality. In certain scenarios, simulator refers to a soft model (GUI model) of the embedded product. For example, if the product under development is a handheld device, to test the functionalities of the various menu and user interfaces, a soft form model of the product with all UI as given in the end product can be developed in software. Soft phone is an example for such a simulator.

Advantages of simulator based debugging.

(i) No Need for Original Target Board

(ii) Simulate I/O Peripherals

(iii) Simulates Abnormal Conditions

Limitations of simulator based debugging

(i) Deviation from Real Behaviour

(ii) Lack of real timeliness

Emulator:

Emulator is hardware device which emulates the functionalities of the target device and allows real time debugging of the embedded firmware in a hardware environment. Emulator' is a self-contained hardware device which emulates the target CPU. The emulator hardware contains necessary emulation logic and it is hooked to the debugging application running on the development PC on one end and connects to the target board through some interface on the other end.

A pure software applications which perform the functioning of a hardware emulator is also called as 'Emulators' (though they are 'Simulators' in operation). The emulator application for emulating the operation of a PDA phone for application development is an example of a 'Software Emulator'. A hardware emulator is controlled by a debugger application running on the development PC. The debugger application may be part of the Integrated Development Environment (IDE) or a third party supplied tool. Most of the IDEs incorporate debugger support for some of the emulators commonly available in the market. The emulators for different families of processors/controllers are different. Figure 13.40 illustrates the different subsystems and interfaces of an 'Emulator' device.

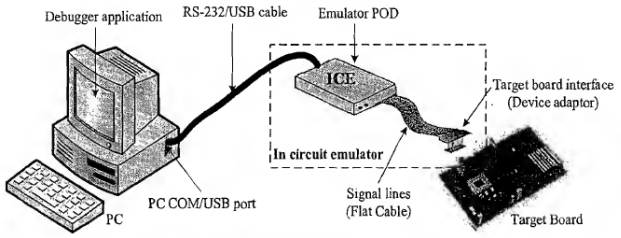


Fig. 13.40 In Circuit Emulator (ICE) Based Target Debugging

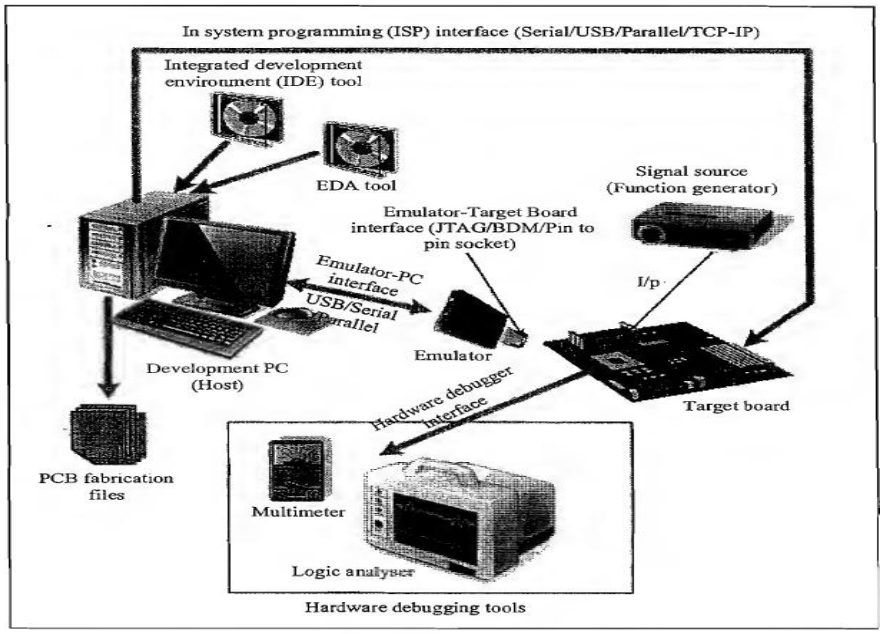
Emulation Device: Emulation device is a replica of the target CPU which receives various signals from the target board through a device adaptor connected to the target board and performs the execution of firmware under the control of debug commands from the debug application. The emulation device can be either a standard chip same as the target processor (e.g. AT89C51) or a Programmable Logic Device (PLD) configured to function as the target CPU.

Emulation Memory: It is the Random Access Memory (RAM) incorporated in the Emulator device. It acts as a replacement to the target board's EEPROM where the code is supposed to be downloaded after each firmware modification. Hence the original EEPROM memory is emulated by the RAM of emulator. This is known as 'ROM Emulation'.

Emulator Control Logic: Emulator control logic is the logic circuits used for implementing complex hardware breakpoints, trace buffer trigger detection, trace buffer control, etc.

Block schematic of IDE Environment for embedded system design:

The various design/development/debug tools employed in embedded system development. A typical embedded system development environment is illustrated in Figure.



The Embedded System Development Environment

As illustrated in the figure, the development environment consists of a Development Computer (PC) or Host, which acts as the heart of the development environment, Integrated Development Environment (IDE) Tool for embedded firmware development and debugging, Electronic Design Automation (EDA) Tool for Embedded Hardware design, An emulator hardware for debugging the target board, Signal sources (like Function generator) for simulating the inputs to the target board, Target hardware.

Integrated environment for developing and debugging the target processor specific embedded firmware. IDE is a software package which bundles a 'Text Editor (Source Code Editor)', 'Cross-compiler (for cross platform development and compiler for same platform development)', 'Linker' and a 'Debugger'. Some IDEs may provide interface to target board emulators, Target processor's/controller's Flash memory programmer, etc. and incorporate other software development utilities like 'Version Control Tool', 'Help File for the Development Language', etc.

IDEs can be either command line based or GUI based. Command line based IDEs may include little or less GUI support. The old version of TURBO C IDE for developing applications in C/C++ for x86 processor on Windows platform is an example for a generic IDE with command line interface. GUI based IDEs provide a Visual Development Environment with mouse click support for each action. Such IDEs are generally known as Visual IDEs. Visual IDEs are very helpful in firmware development. A typical example for a Visual IDE is Microsoft® Visual Studio for developing Visual C++ and Visual Basic programs. Other examples are NetBeans and Eclipse.

IDEs used in embedded firmware development are slightly different from the generic IDEs used for high level language based development for desktop applications. In Embedded Applications, the IDE is either supplied by the target processor/controller manufacturer or by third party vendors or as Open Source. MPLAB is an IDE tool supplied by microchip for developing embedded firmware using their PIC family of microcontrollers. Keil MicroVision3 from Keil software is an example for a third party IDE, which is used for developing embedded firmware for 8051 family microcontrollers. CodeWarrior by Metrowerks is an example of IDE for ARM family of processors.
It should be noted that in embedded firmware development applications each IDE is designed for a specific family of controllers/processors and it may not be possible to develop firmware for all family of controllers/processors using a single IDE.

Different techniques for embedding firmware in the target

Once the firmware starts functioning properly, the next step is embedding the firmware into the target processor/controller. firmware can be embedded using various techniques like Out of System Programming, In System Programming (ISP), etc. Keil IDE provides an In System Programming (ISP) support which is selected at the time of configuring ! the 'Optionsfor Target'. It can be either Keil provided flash programming driver or any third party tool which can be invoked from the IDE. The flash programming makes use of the serial connection established between the Host PC and the target board. The target processor should contain a boot loader or a monitor program which can understand the flash memory programming related commands sent from the IDE. Cross-compile all the source files within the module, link them and generate the binary code using 'Rebuild all targetfiles' option. Download the generated binary file to the target processor's/controller's memory by invoking 'Flash Download' option from the 'Project' tab of the IDE menu. The target board should be powered and interfaced with the host PC where the IDE is running and the connection should be configured properly.
Most of these devices also support in-application programming, allowing the device to modify its program memory under control of the application software. In this way the design can perform on-the-fly software updates while still performing its primary function. Details are presented in the datasheet or user's guide for that particular device.